

The Pattern Programming Language

Programs as Typed Pattern Graphs — Not Text Files That Encode Relationships

David Jean Charlot, PhD Open Interface Engineering, Inc. (openIE)
University of California, Santa Barbara (UCSB) david@openie.dev |
dcharlot@ucsb.edu

February 2026 | Version 1.0

This work is licensed under CC BY-ND 4.0 | Open Access Research

Abstract

Traditional programming languages encode computational intent as syntax that a compiler lowers into machine instructions. The syntax is overhead. Every language — C, Python, Rust, Haskell — expresses the same finite set of computational patterns through different notation. The patterns themselves are invariant: sorting is sorting, graph traversal is graph traversal, matrix multiplication is matrix multiplication. The language barrier is not a barrier of capability but of notation.

We present **Pattern**, a programming language where programs are typed directed acyclic graphs (DAGs) of computational patterns drawn from a universal catalog. Pattern eliminates syntax entirely. A program is a topology — a graph of typed nodes connected by typed edges. Parallelism is discovered from graph structure, not declared by the programmer. Type safety is structural, enforced by edge compatibility. A single Pattern program compiles to 37 target languages through deterministic code generation.

The current catalog contains 800 base patterns across 28 domains with 227,653 type-compatible compositions. An LLM-driven expansion system (the Cartographer) enumerates additional patterns across 48 domains, targeting complete coverage of all known computing. Pattern programs execute in microseconds via deterministic pattern matching, with lazy code generation for newly discovered patterns cached permanently after first use.

Keywords: pattern programming, graph programming, dataflow, type-safe composition, deterministic synthesis, catalog-driven development, LLM-assisted enumeration

1. Introduction

1.1 The Language Barrier Fallacy

Every mainstream programming language is a different notation for the same computational primitives. A merge sort in Python, Rust, Go, and COBOL performs the same algorithm — the only difference is syntax. Decades of language design have produced hundreds of notations, yet the underlying patterns number in the hundreds, not millions. The diversity is in expression, not in computation.

This observation has a concrete consequence: 800 patterns, spanning 28 domains from mathematics to robotics, achieve 100% accuracy on both HumanEval (164/164) [1] and MBPP (974/974) [2] — the two standard benchmarks for code generation. Computing is a bounded endeavor. The patterns are known. What remains is retrieval and composition.

1.2 Programs as Graphs

Pattern programs are not text files. They are typed directed acyclic graphs where nodes represent computational patterns and edges represent typed data flow. This is not a visual metaphor — the graph *is* the program. It is persisted as a binary DAG, validated structurally, and executed by topological traversal.

This eliminates three categories of programming overhead:

1. **Syntax** — there is no syntax to learn, no grammar to parse, no style to debate.
2. **Scheduling** — parallelism is a structural property of the graph, discovered automatically by depth analysis.
3. **Translation** — one graph compiles to 37 target languages through deterministic code generation via the Cortex IR [3].

1.3 The Catalog Is the Language

In traditional languages, power comes from expressiveness — the ability to encode arbitrary computation. In Pattern, power comes from the catalog — the set of known computational patterns. Adding a pattern to the catalog makes every existing and future program richer. Adding syntax to a language makes every existing program more complex.

The Cartographer [4] uses large language models not to generate code but to enumerate and type-sign every known computational pattern across 48 domains of computing. LLMs have already absorbed all of computing’s knowledge implicitly. The Cartographer makes this knowledge explicit, indexed, and composable.

2. Core Philosophy

2.1 All Code Has Already Been Written

The central thesis of Pattern is that programming is database retrieval. Every algorithm, data structure, protocol, and system pattern has been implemented, published, verified, and cataloged — many times over, in many languages. The act of programming is not creation; it is selection and composition.

This is not a philosophical claim. It is an empirical observation: ~800 patterns cover 100% of two independent benchmarks designed to test the breadth of programming ability. The number of distinct computational patterns in all of

computing is large but finite — estimated at 4,000-5,000 across 48 domains.

2.2 Patterns Over Functions

A pattern is not a function. A function is a syntactic construct with an implementation. A pattern is a *unit of computational knowledge* with a typed signature, a domain classification, a keyword set for retrieval, and a description for scaffolding. The implementation is incidental — it can be generated on demand for any target language. What matters is the signature:

```
PatternSig {  
  name: "merge_sort",  
  inputs: [List<Int>],  
  output: List<Int>,  
  category: "sort",  
  keywords: ["sort", "merge", "divide", "conquer", "stable"]  
}
```

Two patterns compose when the output type of one is compatible with an input type of another. The 800-pattern catalog produces 413,449 possible pairs, of which 227,653 (54.9%) are type-compatible. Composition is structural, not syntactic.

2.3 Graphs Over Text

A program is a topology. The nodes are patterns. The edges are typed data channels. The shape of the graph determines what can run in parallel — no annotations, no `async/await`, no thread pools. Two nodes at the same depth in the DAG have no data dependency and can execute simultaneously.

```
Input(List<Int>)  
├─ find_max(List<Int>) → Int  
├─ sum(List<Int>) → Int  
└─ sort(List<Int>) → List<Int>  
    └─ Output(List<Int>)
```

Here, `find_max`, `sum`, and `sort` are independent — the graph topology reveals parallelism that a text representation would obscure.

2.4 Style Is Orthogonal

Correctness is deterministic pattern matching — microseconds. Style is explicit post-processing — the same algorithm rendered with different personality. Pattern separates these concerns completely. A continuous 4D style space (voice, documentation, naming, header) with 8 named presets transforms correct code into code that reads like Kernighan, Knuth, Torvalds, or Dijkstra wrote it. Same correctness. Different soul.

3. Type System

Pattern uses a structural type system with 8 base types. Types are used for port compatibility checking on edges, pattern signature matching, and composition validation.

3.1 Type Definitions

Type	Notation	Description
<code>Int</code>	<code>Int</code>	64-bit signed integer
<code>Float</code>	<code>Float</code>	IEEE 754 double-precision floating-point
<code>Bool</code>	<code>Bool</code>	Boolean value
<code>String</code>	<code>String</code>	UTF-8 text
<code>List<T></code>	<code>List<Int></code>	Homogeneous ordered collection
<code>Tuple<T1,...,Tn></code>	<code>(Int, Float, Bool)</code>	Fixed-width heterogeneous product
<code>Option<T></code>	<code>Option<String></code>	Nullable/optional wrapper
<code>Any</code>	<code>Any</code>	Polymorphic wildcard

3.2 Compatibility Rules

Type compatibility determines whether an edge can connect two ports. The rules are:

1. **Identity:** A type is compatible with itself.
2. **Wildcard:** `Any` is compatible with every type in both directions.
3. **List:** `List<A>` is compatible with `List<B>` if `A` is compatible with `B`.
4. **Option:** `Option<A>` is compatible with `Option<B>` if `A` is compatible with `B`.
5. **Tuple:** `Tuple<A1, ..., An>` is compatible with `Tuple<B1, ..., Bn>` if they have the same arity and each `Ai` is compatible with `Bi`.
6. **Otherwise:** Types are incompatible.

3.3 Dual Representation

The type system exists in two isomorphic forms:

- **SynthType** — the internal representation used by the pattern catalog, composition engine, and synthesis pipeline.
- **GraphType** — the serializable representation used by `ProgramGraph` for binary persistence via bincode.

Bidirectional conversion is lossless: `GraphType::from_synth()` and `GraphType::to_synth()` preserve all structural information including nested generics.

3.4 LLM Type Parsing

The Cartographer parses type annotations from LLM output into `GraphType` values. The parser accepts Python-style notation (`List[Int]`, `Tuple[Int, Float]`, `Option[String]`), angle-bracket notation (`List<Int>`), and common aliases (`i64` → `Int`, `f64` → `Float`, `boolean` → `Bool`, `Vec` → `List`). Unknown types fall back to `Any`.

4. Node Model

Every node in a Pattern program has a unique integer identifier, a kind that determines its semantics, typed input and output ports, and an optional human-readable label.

```
Node {
    id: NodeId,          // u32 – unique within the graph
    kind: NodeKind,      // determines computation
    inputs: Vec<Port>,   // typed input ports
    outputs: Vec<Port>,  // typed output ports
    label: Option<String>
}

Port {
    typ: GraphType,      // port type
    name: String         // human-readable name
}
```

4.1 Node Kinds

Kind	Semantics	Inputs	Outputs
<code>Pattern(id)</code>	Catalog pattern — the computational primitive	From pattern signature	From pattern signature
<code>Literal(value)</code>	Constant value injection into the graph	None	Single port, type inferred from value
<code>Input(index)</code>	Program entry point (argument receiver)	None	Single port, type from program declaration
<code>Output</code>	Program exit point (result emitter)	Single port, type from program declaration	None
<code>SubGraph(program)</code>	Nested program — graphs within graphs	From inner program's input types	From inner program's output type

Pattern nodes are the fundamental computational unit. The string identifier references a pattern in the catalog (static or dynamic). Ports are inferred from the pattern's `PatternSig` when using catalog lookup, or declared explicitly via `pattern_typed()` for dynamic patterns.

Literal nodes inject constants: `Int(42)`, `Float(3.14)`, `Bool(true)`, `Str("hello")`, or `List([1, 2, 3])`. The output port type is inferred from the value.

SubGraph nodes enable hierarchical composition. A `ProgramGraph` is embedded inside another `ProgramGraph`, with ports inherited from the inner graph's declared input and output types. This is Pattern's abstraction mechanism — the equivalent of function definition in text languages.

5. Edge Semantics

Edges are directed, typed data channels between node ports.

```
Edge {
    from_node: NodeId,    // source node
    from_port: usize,     // source port index
    to_node: NodeId,      // target node
    to_port: usize        // target port index
}
```

5.1 Rules

1. **Direction:** Data flows from `from_node:from_port` to `to_node:to_port`. Edges are not bidirectional.
2. **Type safety:** The type of the source port must be compatible with the type of the target port (per Section 3.2).
3. **Fan-out:** Multiple edges may originate from the same output port. This represents data duplication — the same value is sent to multiple consumers.
4. **Fan-in:** Multiple edges may target different input ports of the same node. This represents multi-argument consumption.

5. **No implicit broadcast:** Every data flow must be explicitly wired. There are no implicit connections.
6. **Acyclicity:** The edge set must form a DAG. Cycles are a validation error.

5.2 Convenience Wiring

The builder API provides `connect(from, to)` as shorthand for `edge(from, 0, to, 0)` — connecting the first output port to the first input port. For patterns with single inputs and outputs, this covers the common case.

6. Program Structure

A Pattern program is a complete, self-contained artifact:

```
ProgramGraph {
  version: u32,                // schema version (currently 1)
  name: String,                // program identifier
  description: String,         // human-readable description
  nodes: Vec<Node>,           // all computation nodes
  edges: Vec<Edge>,           // all data flow edges
  input_types: Vec<GraphType>, // ordered program input types
  output_type: GraphType,      // program output type
  temperature: f64,            // synthesis temperature (0.0–1.0)
  metadata: ProgramMetadata    // provenance and discovery tags
}

ProgramMetadata {
  created: u64,                // creation timestamp (Unix millis)
  updated: u64,                // last modification timestamp
  author: Option<String>,      // attribution
  tags: Vec<String>,           // discovery/categorization tags
  provenance: Vec<String>      // ancestry/composition history
}
```

6.1 Program Properties

- **DAG constraint:** The graph must be acyclic. Cycles represent circular data dependencies and are rejected during validation.
- **First-class programs:** Programs can be embedded inside other programs via `SubGraph` nodes. This enables hierarchical composition without a

separate module system.

- **Binary persistence:** Programs serialize to `.gp` files via bincode. A 4-node graph serializes to approximately 200 bytes.
 - **Temperature:** Controls synthesis behavior when patterns are resolved to code. Range 0.0 (deterministic) to 1.0 (exploratory).
-

7. Validation Rules

Validation checks a `ProgramGraph` against a pattern catalog (`Vec<PatternSig>`) and reports all structural errors. A valid program has zero errors.

7.1 Error Types

Error	Condition	Severity
<code>TypeMismatch</code>	Edge connects ports with incompatible types	Fatal — data corruption at runtime
<code>CycleDetected</code>	Graph contains a cycle (lists involved nodes)	Fatal — infinite execution
<code>UnknownPattern</code>	Pattern node references an ID not in the catalog	Fatal — no implementation available
<code>DisconnectedNode</code>	Non-source node has no incoming or outgoing edges	Warning — dead computation
<code>NoOutputNode</code>	Graph has no <code>Output</code> node	Fatal — no result produced
<code>NoInputNode</code>	Graph has no <code>Input</code> or <code>Literal</code> node	Fatal — no data source

7.2 Validation Algorithm

Validation proceeds in three passes:

1. **Pattern resolution:** For each `Pattern` node, look up the pattern ID in the catalog. If not found, emit `UnknownPattern`.

2. **Type checking:** For each edge, compare the source output port type with the target input port type. If incompatible, emit `TypeMismatch`.
3. **Structural checks:** Verify presence of entry nodes (`Input` or `Literal`), presence of `Output` node, and connectivity of all non-source nodes.

Validation against a combined catalog (static 800 + dynamic patterns from the Cartographer) enables programs to reference both established and newly discovered patterns.

8. Execution Model

Pattern programs execute by topological traversal of the DAG. The execution model discovers parallelism from graph structure without any programmer annotation.

8.1 Topological Sort

Execution order is computed via Kahn's algorithm:

1. Build an in-degree map and successor adjacency list from the edge set.
2. Initialize a queue with all nodes having in-degree zero (sources).
3. Process nodes from the queue, decrementing successors' in-degrees.
4. If all nodes are processed, the graph is a valid DAG. If not, the remaining nodes form a cycle.

8.2 Parallel Stage Discovery

After topological sorting, nodes are grouped into parallel execution stages by depth:

1. For each node in topological order, compute its depth as `max(depth of all predecessors) + 1`. Source nodes have depth 0.
2. Group all nodes at the same depth into a single `ExecutionStage`.
3. Nodes within a stage have no mutual data dependencies and can execute simultaneously.

```

ExecutionPlan {
    stages: Vec<ExecutionStage>,    // ordered by depth
    total_nodes: usize,
    max_parallelism: usize           // largest stage width
}

ExecutionStage {
    nodes: Vec<NodeId>               // can execute in parallel
}

```

8.3 Example

Consider a program: `Input → {find_max, sum} → Output`

```

Stage 0: [input_0]                - 1 node (sequential)
Stage 1: [find_max, sum]           - 2 nodes (PARALLEL)
Stage 2: [output]                  - 1 node (sequential)
max_parallelism = 2

```

The parallelism emerges from the topology. No `async`, no `spawn`, no `parallel for` — the graph shape is the schedule.

9. Pattern Catalog

The pattern catalog is the vocabulary of the Pattern language. Its size and coverage determine what programs can be expressed.

9.1 Static Catalog

The base catalog contains 800+ patterns across 28 domains:

Domains: Math, Graph, Dynamic Programming, String, Tree, Search, Sort, Machine Learning, Distributed Systems, Database, Networking, Cryptography, Imaging, Signal Processing, Control Systems, Quantum Computing, Physics, Bioinformatics, Formal Verification, Finance, Optimization, Compression, Error Correction, Graphics, Robotics, Information Retrieval, Audio Processing, GIS.

Each pattern is represented as a `PatternSig`:

```

PatternSig {
  name: String,           // canonical identifier (e.g.,
  "merge_sort")
  inputs: Vec<SynthType>, // ordered input types
  output: SynthType,      // output type
  category: String,       // domain classification
  keywords: Vec<String>   // retrieval keywords
}

```

9.2 Composition Space

From 800 base patterns, the composition engine generates:

- **413,449** total possible pairs (patterns x patterns)
- **227,653** type-compatible pairs (54.9%) — 44.9% eliminated by type checking
- **71** pre-validated domain recipes via the NCO doctrine layer

Type-safe composition is guaranteed at graph construction time. If an edge validates, the composition is sound.

9.3 Pattern Resolution Chain

When a Pattern program is executed, each pattern node is resolved to executable code through a layered lookup:

Layer	Mechanism	Latency	Confidence
Lo	Learned memory (cached prior results)	1 us	0.95
Enlisted	Static catalog pattern matching	23 us avg	0.70-0.90
NCO	Domain recipe composition	~100 us	0.70-0.90
L2	Pure deterministic synthesis	ms	0.30-0.80
L3	LLM-guided hybrid synthesis	1-30s	0.50-0.80

First success wins. Lo absorbs every L2/L3 result, so each pattern is generated at most once. Second execution: microseconds.

10. Dynamic Patterns and the Cartographer

The static catalog covers algorithms and data structures. System-level software — operating systems, compilers, databases, network stacks — requires thousands more patterns. The Cartographer bridges this gap.

10.1 Architecture

```
LLM (knows all patterns implicitly)
  → Cartographer (enumerate + type-sign + deduplicate)
    → DynamicCatalog (persistent runtime registry)
      → CodeFactory (lookup: Static → Dynamic → L3)
        → L0 absorbs result (microsecond retrieval forever)
```

The Cartographer uses an LLM Provider to enumerate computational patterns within a domain. For each domain, it issues a structured prompt requesting pattern names, typed signatures, keywords, and descriptions. The LLM response is parsed into `DiscoveredPattern` entries, deduplicated against the static catalog, and stored in the `DynamicCatalog`.

10.2 Dynamic Catalog

```
DynamicCatalog {
  patterns: Vec<DiscoveredPattern>,
  by_name: HashMap<String, usize>,          // O(1) name lookup
  by_keyword: HashMap<String, Vec<usize>>, // keyword index
  by_domain: HashMap<String, Vec<usize>>    // domain index
}
```

Keyword search uses Jaccard similarity: $|A \cap B| / |A \cup B|$. Results are sorted by score and filtered by a configurable threshold.

Dynamic pattern names are prefixed with `dyn:` when converted to `PatternSig` entries, preventing collision with the static 800.

10.3 Expanded Domain Coverage

The Cartographer targets 48 domains — the original 28 plus 20 system-level domains:

New domains: OS Kernel, Filesystem, Device Driver, Window System, Shell, Compiler Frontend, Compiler Backend, Virtual Machine, Container, Package Manager, Build System, Text Editor, Version Control, Web Server, Web Client, Testing, Logging, Monitoring, Serialization, Concurrency.

Target: ~ 100 patterns per domain $\times$ 48 domains = $\sim 4,800$ patterns. With deduplication against the static 800, this yields approximately 3,500-4,000 net new patterns covering the entirety of known computing.

10.4 Lazy Code Generation

The Cartographer discovers signatures, not implementations. Code is generated lazily:

1. A Pattern program references `dyn:round_robin_scheduler`.
2. At execution time, Lo has no cached result. The OODA engagement loop escalates to L3.
3. L3 generates code using the pattern's description as a scaffold.
4. Lo absorbs the result. The pattern is now available in microseconds.
5. Every subsequent execution of any program referencing this pattern hits Lo.

This is a one-time cost per pattern, amortized across all future programs.

11. Code Generation

A single Pattern program compiles to 37 target languages through the Cortex IR pipeline [3].

11.1 Pipeline

Pattern Node $\rightarrow$ IrBridge $\rightarrow$ Cortex IR $\rightarrow$ CodeGen $\rightarrow$ Target Language

1. **IrBridge** maps pattern identifiers to `PatternInfo` structures containing semantic argument names and IR generation functions.

2. **Cortex IR** is a register-based intermediate representation with `arg{i}` for function arguments, `var_{i}` for virtual registers, and a `pc` state machine for control flow.
3. **CodeGen** lowers Cortex IR to target language syntax.

11.2 Supported Languages (37)

Systems: Rust, C, C++, Zig, D, V, Ada **JVM/CLR:** Java, C#, Kotlin, Scala, Groovy **Legacy:** COBOL, Fortran **Scripting:** Python, JavaScript, TypeScript, Ruby, PHP, Perl, Lua, Bash **Functional:** Haskell, OCaml, F#, Elixir, Clojure **Modern:** Go, Swift, Dart, Julia, R, Nim, Mojo **Emerging:** AssemblyScript, Grain, MoonBit

The same graph topology produces correct, idiomatic code in each target.
Average synthesis time: 23 microseconds per pattern.

12. Persistence Format

Pattern programs use a compact binary serialization format.

12.1 Binary Format

- **Encoding:** bincode — a compact binary serialization crate for Rust [5].
- **File extension:** `.gp` (Graph Program).
- **All types** derive `Serialize` and `Deserialize`.
- **API:** `ProgramGraph::save(path)` and `ProgramGraph::load(path)`.

12.2 Characteristics

- **Compact:** A 4-node sequential pipeline serializes to approximately 200 bytes.
- **Schema versioned:** The `version` field enables future format evolution.
- **Not human-readable:** The binary format prioritizes size and speed over readability. The `Display` implementation on `ProgramGraph` provides human-readable output for inspection.

The `DynamicCatalog` also persists via bincode (`.bin` files), enabling the Cartographer's discoveries to survive across sessions.

13. Style System

Pattern separates correctness from presentation through an explicit style engine [6].

13.1 4D Style Space

Every piece of generated code exists at a coordinate in a continuous 4-dimensional style space:

Axis	Range	Meaning
<code>voice</code>	0.0-1.0	Comment density: silent → terse → balanced → verbose → poetic
<code>doc_level</code>	0.0-1.0	Documentation depth: none → minimal → standard → academic
<code>naming</code>	0.0-1.0	Identifier verbosity: single-char → short → full → mathematical
<code>header</code>	0.0-1.0	Header weight: none → minimal → standard → full attribution

13.2 Named Presets

8 presets map to coordinates in the style space:

Preset	voice	doc	naming	header	Character
kernighan	0.25	0.33	0.00	0.33	Terse, minimal, single-char names
knuth	0.75	1.00	0.66	1.00	Verbose, academic, fully documented
torvalds	0.25	0.00	0.33	0.33	Sparse, no docs, short names
dijkstra	0.50	0.66	1.00	0.66	Balanced voice, mathematical names
pythonista	0.50	0.66	0.66	0.66	PEP-8 balanced
rustacean	0.50	0.33	0.66	0.33	Idiomatic Rust
hacker	0.25	0.00	0.00	0.00	Minimal everything
clean	0.50	0.66	0.33	0.66	Uncle Bob balanced

Custom profiles can be created at any coordinate. Two profiles can be interpolated: `blend(&a, &b, 0.5)` produces the midpoint style.

13.3 Transforms

Five transforms are applied in order to convert raw synthesized code into styled code:

1. **strip_noise** — remove auto-generated artifacts
2. **rename_args** — map arguments to style-appropriate names
3. **adjust_voice** — add or remove comments based on voice level
4. **restyle_header** — adjust file/function header depth
5. **inject_docs** — add documentation blocks based on `doc_level`

13.4 GPU Acceleration

A Metal compute kernel searches the 4D style space in parallel. Given a style objective (target metrics extracted from reference code), the kernel evaluates 160,000 candidate profiles in approximately 10ms, identifying the optimal style coordinate analytically without string manipulation on the GPU.

14. Builder API

Programs are constructed through a fluent builder:

```
let catalog = build_pattern_signatures();
let mut builder = ProgramBuilder::new("sort_and_dedup");

let input = builder.input(GraphType::List(Box::new(GraphType::Int)),
    "numbers");
let sort = builder.pattern("merge_sort", &catalog);
let dedup = builder.pattern("remove_duplicates", &catalog);
let output = builder.output(GraphType::List(Box::new(GraphType::Int)));

builder.connect(input, sort);
builder.connect(sort, dedup);
builder.connect(dedup, output);

let program = builder.build();
```

14.1 Builder Methods

Node creation (returns `NodeId`): - `input(typ, name)` — program input port
- `pattern(id, catalog)` — catalog pattern with inferred ports -
`pattern_typed(id, inputs, output)` — explicit types (for dynamic patterns) -
`literal(value)` — constant value injection - `subgraph(program)` — nested
program - `output(typ)` — program output port

Edge creation: - `connect(from, to)` — connect port `o` to port `o` -
`edge(from, from_port, to, to_port)` — explicit port wiring

Configuration (chainable): - `description(text)` — set program description -
`temperature(t)` — set synthesis temperature (0.0-1.0) - `author(name)` — set
author attribution - `tag(label)` — add discovery tag

Finalization: - `build()` — consume builder, return `ProgramGraph`

15. Examples

15.1 Sequential Pipeline

Sort a list of integers and remove duplicates:

```
let mut b = ProgramBuilder::new("sort_and_dedup");
let inp = b.input(GraphType::List(Box::new(GraphType::Int)), "data");
let srt = b.pattern("merge_sort", &catalog);
let ded = b.pattern("remove_duplicates", &catalog);
let out = b.output(GraphType::List(Box::new(GraphType::Int)));
b.connect(inp, srt);
b.connect(srt, ded);
b.connect(ded, out);
let program = b.build();
```

Topology: `input` → `merge_sort` → `remove_duplicates` → `output`

Execution plan: 4 stages, max parallelism = 1 (fully sequential) **Validation:** 0 errors against static catalog

15.2 Parallel Fork-Join

Compute multiple statistics from one input simultaneously:

```
let mut b = ProgramBuilder::new("parallel_stats");
let inp = b.input(GraphType::List(Box::new(GraphType::Int)), "data");
let mx = b.pattern("find_max", &catalog);
let sm = b.pattern("sum", &catalog);
let out1 = b.output(GraphType::Int);
b.connect(inp, mx);
b.connect(inp, sm);
// Both mx and sm feed downstream (simplified)
```

Topology: `input` → {`find_max`, `sum`} (fan-out from single input) **Execution plan:** Stage 0: [`input`], Stage 1: [`find_max`, `sum`] (PARALLEL) **max_parallelism:** 2 — discovered from topology, not declared

15.3 Dynamic Pattern (Cartographer-Discovered)

Use a pattern that does not exist in the static 800:

```
let mut b = ProgramBuilder::new("os_scheduler");
let procs = b.input(GraphType::List(Box::new(GraphType::Int)),
    "processes");
let quantum = b.input(GraphType::Int, "time_quantum");
let sched = b.pattern_typed(
    "dyn:round_robin_scheduler",
    vec![GraphType::List(Box::new(GraphType::Int)), GraphType::Int],
    GraphType::List(Box::new(GraphType::Int)),
);
let out = b.output(GraphType::List(Box::new(GraphType::Int)));
b.connect(procs, sched);
b.connect(quantum, sched);
b.connect(sched, out);
let program = b.build();
```

Topology: {processes, time_quantum} → dyn:round_robin_scheduler → output **Resolution:** L3 generates code on first execution; LO caches permanently. **The pattern was discovered by the Cartographer, not hand-coded.**

15.4 Nested SubGraph

Embed a reusable inner program as a node:

```

// Inner: sort pipeline
let mut inner_b = ProgramBuilder::new("sort_pipeline");
let ii = inner_b.input(GraphType::List(Box::new(GraphType::Int)),
    "data");
let is = inner_b.pattern("merge_sort", &catalog);
let io = inner_b.output(GraphType::List(Box::new(GraphType::Int)));
inner_b.connect(ii, is);
inner_b.connect(is, io);
let inner = inner_b.build();

// Outer: uses inner as a subgraph node
let mut outer_b = ProgramBuilder::new("outer_program");
let oi = outer_b.input(GraphType::List(Box::new(GraphType::Int)), "raw");
let sub = outer_b.subgraph(inner);
let od = outer_b.pattern("remove_duplicates", &catalog);
let oo = outer_b.output(GraphType::List(Box::new(GraphType::Int)));
outer_b.connect(oi, sub);
outer_b.connect(sub, od);
outer_b.connect(od, oo);
let outer = outer_b.build();

```

Topology: input → [sort_pipeline] → remove_duplicates → output **The SubGraph node encapsulates the entire sort pipeline as a single node in the outer graph.**

16. Formal Grammar

The structural grammar of a Pattern program in BNF-like notation:

```

Program      ::= Name Version Nodes Edges InputTypes OutputType
Temperature Metadata
Name         ::= STRING
Version      ::= UINT32
Nodes        ::= Node*
Edges        ::= Edge*
InputTypes   ::= Type*
OutputType   ::= Type
Temperature  ::= FLOAT64

Node         ::= NodeId NodeKind Ports Ports Label
NodeId       ::= UINT32
NodeKind     ::= 'Pattern' '(' STRING ')'
              | 'Literal' '(' LiteralValue ')'
              | 'Input' '(' UINT ')'
              | 'Output'
              | 'SubGraph' '(' Program ')'

Ports        ::= Port*
Port         ::= Type STRING
Label        ::= STRING | EMPTY

Edge         ::= NodeId UINT NodeId UINT    -- (from_node, from_port,
to_node, to_port)

Type         ::= 'Int' | 'Float' | 'Bool' | 'String'
              | 'List' '<' Type '>'
              | 'Tuple' '<' Type (',' Type)* '>'
              | 'Option' '<' Type '>'
              | 'Any'

LiteralValue ::= INT64 | FLOAT64 | BOOL | STRING
              | '[' LiteralValue (',' LiteralValue)* ']'

Metadata     ::= Created Updated Author Tags Provenance
Created      ::= UINT64                -- Unix milliseconds
Updated      ::= UINT64
Author       ::= STRING | EMPTY
Tags         ::= STRING*
Provenance   ::= STRING*

```

Constraint: The edge set must form a directed acyclic graph (DAG).

17. Comparison with Prior Art

17.1 Dataflow Programming (Dennis & Misunas, LabVIEW, Simulink)

The dataflow model of computation, introduced by Dennis and Misunas [8], represents programs as directed graphs where nodes fire when all inputs are available. LabVIEW [9] and Simulink [10] commercialized this model for instrumentation and signal processing respectively, with visual graph editors where users wire arbitrary functions together. Pattern shares the dataflow execution model — nodes fire when predecessor edges deliver data — but differs fundamentally in the nature of the nodes. Dataflow graphs wire arbitrary user-defined functions, including potentially incorrect ones. Pattern graphs wire *cataloged computational knowledge* — pre-validated, typed, keyword-indexed patterns drawn from a universal registry. Correctness is guaranteed by catalog membership, not by testing. Furthermore, Pattern’s typed edge system enforces structural type compatibility that LabVIEW’s polymorphic wires do not.

17.2 Flow-Based Programming (Morrison)

Morrison’s flow-based programming (FBP) [11] proposes that applications should be defined as networks of “black box” processes exchanging data across predefined connections. FBP shares Pattern’s philosophy that programs are graphs, not text. However, FBP processes are hand-written components with arbitrary internal complexity. Pattern’s nodes are atomic patterns from a typed catalog — the internal complexity is resolved by the synthesis chain (Enlisted $\rightarrow$ NCO $\rightarrow$ L2 $\rightarrow$ L3), not by the programmer. FBP also lacks Pattern’s type system: connections carry untyped “information packets” whereas Pattern edges carry structurally typed data.

17.3 Pattern Languages (Alexander, Gang of Four)

Christopher Alexander’s *A Pattern Language* [12] introduced the concept of reusable solution patterns in architecture. The Gang of Four [13] adapted this to object-oriented software design with 23 design patterns (Factory, Observer,

Strategy, etc.). These pattern languages are descriptive — they document recurring solutions that humans apply manually. Pattern (the programming language) is prescriptive — patterns are executable, typed, composable computational units retrieved by the system automatically. Where Alexander cataloged spatial relationships and Gamma et al. cataloged class relationships, Pattern catalogs computational relationships with formal type signatures.

17.4 Functional Composition (Haskell, F#, Elixir)

Functional languages compose functions through syntax: `f . g . h`. Pattern composes patterns through topology: edges connecting typed ports. The advantage is that Pattern’s composition reveals parallelism structurally — two branches in a graph with no shared edges are independent. In a functional pipeline, parallelism requires explicit annotation or compiler inference from effect analysis [14]. Pattern also separates the composition structure (the graph) from the implementation language — the same graph generates code in 37 languages, while a Haskell composition is bound to Haskell.

17.5 Pipeline Languages (Unix Pipes, dplyr, Apache Beam)

The Unix pipe model [15] establishes sequential composition of text-processing programs. Modern pipeline frameworks like Apache Beam [16] and dplyr [17] extend this to typed, parallel data pipelines. Pattern programs generalize pipelines: a pipeline is a degenerate Pattern program where every stage has in-degree and out-degree of 1. Pattern programs are multi-dimensional DAGs with fan-out and fan-in, enabling topologies that pipelines cannot express (parallel branches, diamond merges, multi-input aggregations).

17.6 Visual Programming (Scratch, Node-RED, Unreal Blueprints)

Visual programming systems represent programs as visual graphs that are *rendered* from an underlying text or AST representation [18]. Scratch uses blocks, Node-RED uses wired nodes, Unreal Blueprints use visual scripting. Pattern’s graph is not a visualization — it is the program itself. The binary `.gp`

format stores the graph directly. There is no underlying text representation being visualized. This distinction matters: visual programming adds a visual layer on top of a textual substrate, while Pattern eliminates the textual substrate entirely.

17.7 Program Synthesis (Gulwani, Polozov, Singh)

Program synthesis generates programs from specifications — input/output examples, logical constraints, or natural language descriptions [19]. Systems like FlashFill [20] and recent neural program synthesis [21] produce code from scratch. Pattern does not synthesize programs from specifications. It composes pre-validated patterns according to typed edges. Synthesis in Pattern occurs only at the pattern level (L2 and L3 generate individual pattern implementations), not at the graph level. The graph — the program — is constructed by the programmer or by the Cartographer, not synthesized from examples.

17.8 LLM Code Generation (Copilot, Codex, Claude)

LLM code generators [22] produce text from natural language prompts. Each generation is stochastic, potentially incorrect, and expensive (seconds, dollars). Pattern does not generate code from scratch. It retrieves pre-validated patterns and composes them according to typed edges. Correctness is guaranteed by construction. Cost per pattern: 23 microseconds. The LLM's role in Pattern is catalog population via the Cartographer [4], not runtime code generation.

18. Future Work

18.1 Self-Hosting

Pattern programs that construct Pattern programs — the language's `ProgramBuilder` expressed as Pattern nodes. This requires the catalog to contain meta-patterns: `create_node`, `add_edge`, `validate_graph`, `topological_sort`. The Cartographer can enumerate these from a `graph_meta` domain.

18.2 Live Graph Editing

Runtime topology modification — adding, removing, or rewiring nodes in a running program. This enables adaptive programs that restructure themselves based on runtime metrics or environmental changes.

18.3 Distributed Execution

Partitioning execution stages across machines. Nodes within a parallel stage can be dispatched to different compute nodes. The execution plan’s stage boundaries are natural partition points — no data flows between nodes within a stage.

18.4 Hardware-Aware Dispatch

Matching pattern characteristics to hardware capabilities. A `matrix_multiply` pattern dispatches to GPU. A `hash_lookup` pattern stays on CPU. A `neural_inference` pattern targets NPU. The dispatch decision is per-node, informed by the Metabolic Cascade telemetry system [7].

References

- [1] M. Chen et al., “Evaluating Large Language Models Trained on Code,” *arXiv preprint arXiv:2107.03374*, 2021. (HumanEval benchmark)
- [2] J. Austin et al., “Program Synthesis with Large Language Models,” *arXiv preprint arXiv:2108.07732*, 2021. (MBPP benchmark)
- [3] D. J. Charlot, “Cortex: The Universal AI Compiler,” *AGI-Wisdom Technical Report*, January 2026.
- [4] D. J. Charlot, “The Cartographer: LLM-Driven Pattern Catalog Expansion,” *AGI-Wisdom Technical Report*, February 2026.
- [5] D. Tolnay, “`bincode` — A compact binary serialization format,” *crates.io*, 2024.
- [6] D. J. Charlot, “Continuous 4D Code Personality: The Style Engine,” *AGI-Wisdom Technical Report*, January 2026.

- [7] D. J. Charlot, “Metabolic Cascade Inference: Hardware-Aware Adaptive Routing for Energy-Efficient AI,” *AGI-Wisdom Technical Report*, January 2026.
- [8] J. B. Dennis and D. P. Misunas, “A Preliminary Architecture for a Basic Data-Flow Processor,” *Proceedings of the 2nd Annual Symposium on Computer Architecture (ISCA)*, pp. 126–132, 1975.
- [9] National Instruments, “LabVIEW: A Graphical Programming Environment,” *National Instruments Technical Documentation*, 1986–2024.
- [10] MathWorks, “Simulink: Simulation and Model-Based Design,” *mathworks.com*, 2024.
- [11] J. P. Morrison, *Flow-Based Programming: A New Approach to Application Development*, 2nd ed. CreateSpace Independent Publishing, 2010.
- [12] C. Alexander, S. Ishikawa, and M. Silverstein, *A Pattern Language: Towns, Buildings, Construction*. Oxford University Press, 1977. ISBN 978-0-19-501919-3
- [13] E. Gamma, R. Helm, R. Johnson, and J. Vlissides, *Design Patterns: Elements of Reusable Object-Oriented Software*. Addison-Wesley, 1994. ISBN 978-0-201-63361-0
- [14] S. Peyton Jones and S. Marlow, “Secrets of the Glasgow Haskell Compiler *Inliner*,” *Journal of Functional Programming*, vol. 12, no. 5, pp. 393–434, 2002.
- [15] D. McIlroy, “A Research UNIX Reader: Annotated Excerpts from the Programmer’s Manual, 1971-1986,” *Bell Labs Technical Report CSTR 139*, 1987.
- [16] T. Akidau et al., “The Dataflow Model: A Practical Approach to Balancing Correctness, Latency, and Cost in Massive-Scale, Unbounded, Out-of-Order Data Processing,” *Proceedings of the VLDB Endowment*, vol. 8, no. 12, pp. 1792–1803, 2015. (Apache Beam foundation)
- [17] H. Wickham et al., “dplyr: A Grammar of Data Manipulation,” *R package*, CRAN, 2024.
- [18] M. Burnett, “Visual Programming,” *Encyclopedia of Electrical and Electronics Engineering*, Wiley, 1999.
- [19] S. Gulwani, O. Polozov, and R. Singh, “Program Synthesis,” *Foundations and Trends in Programming Languages*, vol. 4, no. 1-2, pp. 1–119, 2017.

- [20] S. Gulwani, “Automating String Processing in Spreadsheets Using Input-Output Examples,” *POPL*, pp. 317–330, 2011. (FlashFill)
- [21] M. Balog et al., “DeepCoder: Learning to Write Programs,” *ICLR*, 2017.
- [22] M. Chen et al., “Evaluating Large Language Models Trained on Code,” *OpenAI Codex*, 2021.